

CSE220
Online on Convolution

Subsections: A1, A2

Total Marks: 10

Total Time: 40 Minutes

A discrete-time system is **LTI** only if it is *both* linear *and* time-invariant:

$$\textbf{Linearity: } T\{a x_1(n) + b x_2(n)\} = a T\{x_1(n)\} + b T\{x_2(n)\}$$

$$\textbf{Time-invariance: } T\{x(n - k)\} = y(n - k), \quad \text{where } y(n) = T\{x(n)\}$$

In this online, you will reuse your `DiscreteSignal` and `LTISystem` classes from the offline to write *generic* tests for these two properties, and apply them to two different systems.

Given

$$x_1[n] : \quad n = -2 \dots 2, \quad \text{values} = [1, 0, 2, -1, 3]$$

$$x_2[n] : \quad n = -1 \dots 3, \quad \text{values} = [2, -3, 0, 1, 1]$$

$$a = 2, \quad b = -3, \quad k = 3$$

$$h[n] : \quad n = 0 \dots 2, \quad h[0] = 1, \quad h[1] = 0.5, \quad h[2] = 0.25$$

System A is the LTI system with impulse response $h[n]$ above.

System B is a *time-varying amplifier* defined directly on samples:

$$y_B(n) = n \cdot x(n).$$

Tasks

- Implement `test_linearity(apply_system, x1, x2, a, b)`, which returns the maximum absolute difference between $T\{ax_1 + bx_2\}$ and $aT\{x_1\} + bT\{x_2\}$. This function must work for *any* callable `apply_system` that maps a `DiscreteSignal` to a `DiscreteSignal`, it must not assume the system is an `LTISystem`.
- Implement `test_time_invariance(apply_system, x, k)` the same way, returning the maximum absolute difference between $T\{x(n - k)\}$ and $y(n - k)$.
- Implement `system_b(input_signal)`, a plain function computing $y_B(n) = n \cdot x(n)$ using only `DiscreteSignal` operations.
- Run both tests on **System A** (via `system_a.output`) and on **System B** (via `system_b`), using the given x_1, x_2, a, b, k . Print all four max-difference values.
- In a `print` statement, state which of the two properties System B fails.

Submission

Rename your python file as 2305xxx.py and submit in the moodle.

Note: passing a function as a parameter

In Python, a function is just a value, like a number or a string. You can give it a name, pass it into another function, and that other function can call it later using parentheses. This has nothing to do with signals, it is just to show the pattern you will use.

```
def double(n):
    return n * 2

def square(n):
    return n * n

def apply_and_print(func, value):
    # "func" is whatever function was passed in. Calling func(value)
    # calls THAT function.
    result = func(value)
    print(f"{result}")

apply_and_print(double, 5)    # prints: 10
apply_and_print(square, 5)    # prints: 25
```